RTB Cycle 15 – Accelerating Backend Application Development with AI Code Assistant (Basic)

IMPETUS

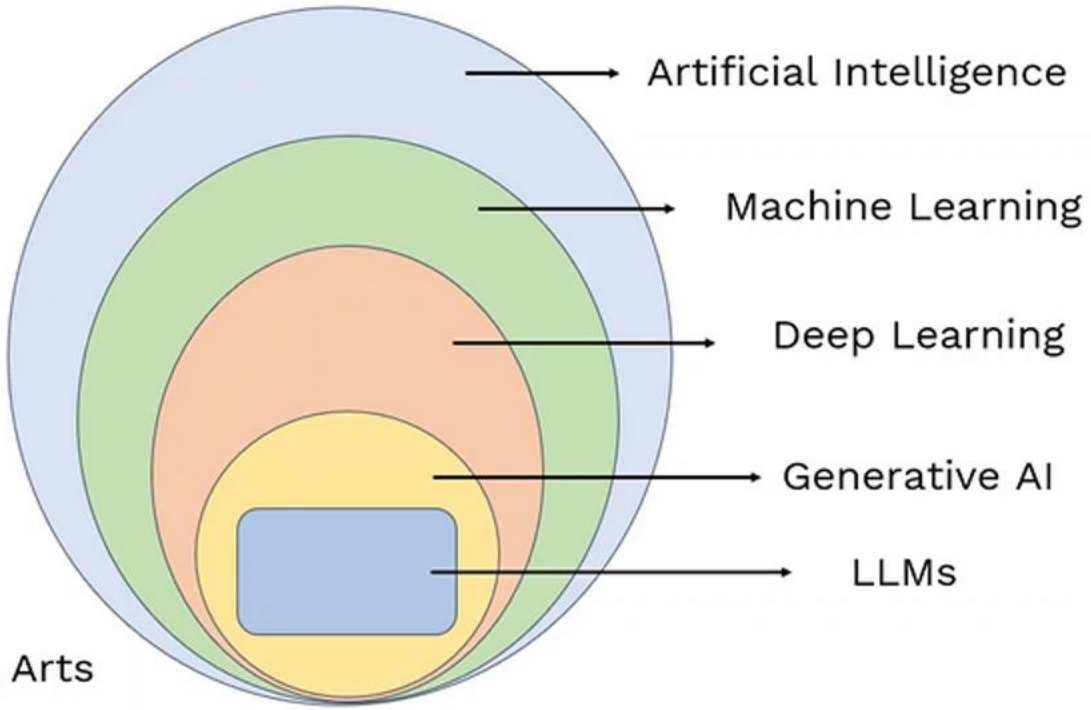
Generative AI

- Generative AI refers to models that **create new data** similar to the data they were trained on. Instead of just predicting labels, they generate content such as text, images, audio, or even code.
- Generates new data trained on huge dataset.
- **Example:**
- ChatGPT generating human-like text responses.
- DALL·E creating images from text prompts.

IMPETUS

What is Generative AI ?

- Branch of AI that can create new content (text, images, audio, video, code) instead of just analyzing data
- Powered by Large Language Models trained on massive datasets
- Enables faster decision making, personalization & innovation at scale
- Real world impact across domains, Tech-IT, Healthcare, Finance, Education & Arts

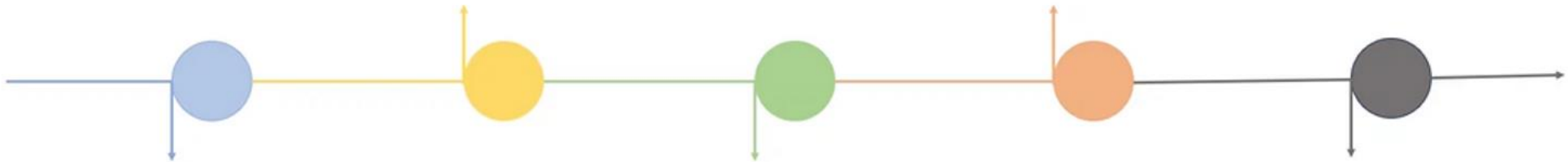


IMPETUS

Evolution of GenAI

Word Embeddings(2013-2017)
- Word2Vec, GloVe
- Words captured in vectors with relationships

Language Models(2018-2023)
- GPT, BERT, Llama, Gemini
- Models that generate, summarize at scale



Early NLP(1990s- 2010s)
- Rule based systems, TF-IDF
- Machines could read words but couldn't understand context

Transformers(2017)
- Attention Mechanism
- **Game Changer :** Models focus on relevant context

RAG Models(2021-today)
- Vector Databases
- LLMs with the power of knowledge base
- Agents

From Natural Language Processing to LLMs

IMPETUS

Why are Generative Models Required?

- Generative models are needed because:
- They **simulate real-world data** for tasks like data augmentation.
- Enable **creative applications** (art, music, design).
- Useful in **unsupervised learning** where labeled data is scarce.
- Power **personalization** (e.g., recommendation systems).
- **Example:**
- Generating synthetic medical images to train diagnostic models when real data is limited.

IMPETUS

Key Applications of Generative Models

- **Text:** Chatbots, content creation.
- **Image:** Art, design, medical imaging.
- **Audio:** Music generation, voice cloning.
- **Code:** AI-assisted programming.
- **Synthetic Data:** Privacy-preserving datasets.

IMPETUS

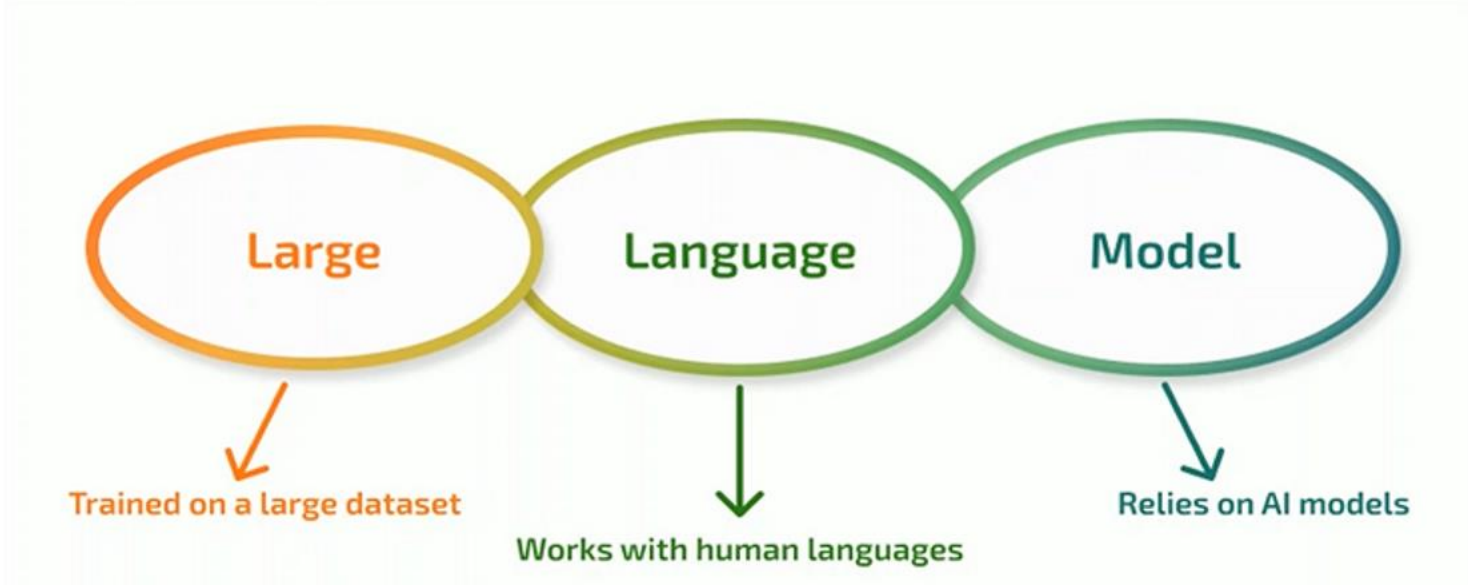
Why GenAI for Developers ?

- **Boost Productivity**: Faster coding, fewer repetitive tasks.
- **Error Reduction**: AI helps catch common mistakes early.
- **Learning Aid**: Helps understand unfamiliar frameworks and languages.
- **Improved Code Quality**: Suggests best practices & optimized solutions.
- **Rapid Prototyping**: Quickly create scaffolding, tests, documentation.
- **Focus on Logic**: Lets developers spend more time on architecture and problem-solving.

IMPETUS

LLM (Large Language Model)

LLM is AI programs that understand and generate human-like text by being trained on massive amounts of data. They have learned patterns and information from the data they were trained on.



IMPETUS

Large Language Models

- Foundation Models => Pre-trained models => Huge Data
- Open AI and Hugging Face were the first companies which came with the concept of LLM Models.
- **Fine Tuning**
- Fine-tuning is a **technique used to adapt a pre-trained model to a specific task or domain** without training it from scratch. It's one of the most important steps in modern AI development because large models like GPT, BERT, or Stable Diffusion are already trained on massive datasets, and fine-tuning allows you to customize them for your needs efficiently.
- **Adjusting the weights of a pre-trained model using a smaller, task-specific dataset.**
- **Why?**
- Training from scratch is expensive (requires huge data and compute).
- Pre-trained models already understand general language or image patterns.
- Fine-tuning makes them **specialized** for your domain (e.g., legal, medical, e-commerce).

IMPETUS

LLM Families & Tools

- Major LLM families developers commonly use:
- **GPT Family (OpenAI / GitHub Copilot)**
- **Claude (Anthropic)**
- **Gemini (Google)**
- **LLaMA (Meta)**
- **Mistral AI**
- **Qwen (Alibaba)**
- **Local LLM Platforms**: Ollama, TabbyML, LM Studio

IMPETUS

Popular LLMs Used in Coding Assistants

- **GPT (GitHub Copilot)**
- Best-in-class for code generation.
- Deep integrations with VS Code, JetBrains.
- Strong reasoning and refactoring abilities.
- **Claude**
- Excellent for clean, structured code & documentation.
- Used in some IDE-integrated plugins.
- **Gemini / Google Models**
- Strong multimodal understanding (code + images).
- Often used in Android/Flutter tooling.
- **LLaMA, Mistral, Qwen**
- Lightweight and efficient for **local/offline** setups.
- Great for privacy-sensitive environments.

IMPETUS

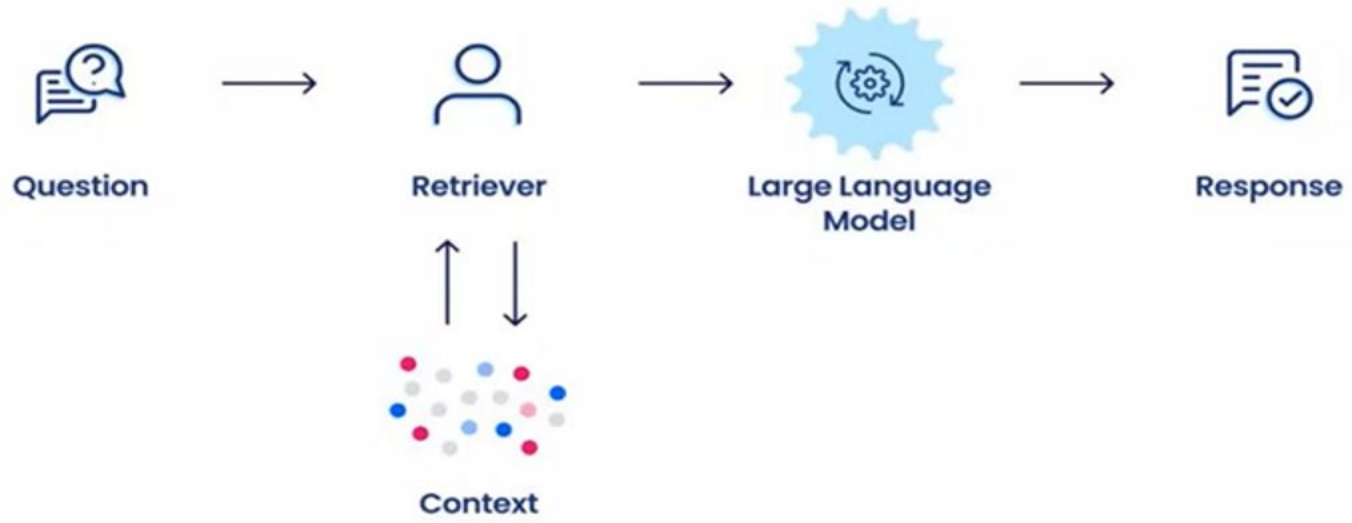
Online vs Local Coding Assistants

- **Online Tools (Cloud-Powered)**
- **GitHub Copilot**
- **Cursor AI**
- **Windsurf**
- ✓ Better accuracy
- ✓ Latest models
- ✓ High reasoning power
- ✗ Requires internet
- ✗ May have data-security concerns
- **Local Tools (Offline or Self-Hosted)**
- **CodeWhisperer (offline mode)**
- **Ollama + CodeGPT**
- **TabbyML**
- ✓ Full privacy
- ✓ Customizable
- ✓ No internet required
- ✗ Slower on low-power machines
- ✗ Requires setup and hardware (RAM/GPU)

IMPETUS

Introduction to RAG (Retrieval Augmentation Generation)

RAG is an AI framework that helps LLMs deliver more accurate and relevant responses by allowing models to access data not included in their training. RAG is a setup that lets an LLM look up relevant documents before answering.



IMPETUS

RAG Continued..

- RAG is an approach in AI that combines a retrieval mechanism with a generative model.
- It is an AI technique that combines **two powerful components**:
- **Retrieval**:
  - **The model searches for relevant information from an external knowledge source (like a database, documents, or the web).**
  - **This ensures the model has access to up-to-date and factual information instead of relying only on its pre-trained knowledge.**
- **Generation**:
  - **After retrieving the relevant data, the model uses a language generation model (like GPT) to create a natural, coherent response.**
  - **The generation step uses both the retrieved context and its own reasoning ability.**
- **In short:**
- RAG = **Search + Generate**
- It's like asking a smart assistant that first looks up the latest info and then explains it in a human-like way.
-
- **How RAG Works (Step-by-Step):**
- **User asks a question** → e.g., "What is the latest iPhone model?"
- **Retriever searches** → Finds relevant documents from a knowledge base.
- **Generator creates answer** → Combines retrieved info + language model to give a clear response.

IMPETUS

Prompt Engineering

- The skill of writing clear instructions for an AI model to produce correct, high-quality outputs.
- Key Components:
- **Role** → Define what the AI should behave as ("Act as a senior Python developer").
- **Task** → What you want done (generate code, fix bug, optimize logic).
- **Context** → Provide background (framework, architecture, constraints).
- **Examples** → Improve accuracy by showing patterns.
- **Constraints** → Specify style, libraries, formats.

IMPETUS

What is a Prompt ?

- A **prompt** is the input or instruction you give to an AI model to generate a response.
- It can be:
- A question
- A command
- A description
- A piece of code
- A task request
- Any text that guides the AI
- **Think of a prompt as the “instruction + context” you give to the AI to get the output you want.**

IMPETUS

Types of Prompts

1. Instruction Prompts

- Direct commands telling AI **what to do**.
- **Example:**
- “Write a Python function to sort a list in descending order.”

2. **Contextual Prompts**

- Include background information to guide the AI.
- **Example:**
- “I’m building a React website with Firebase authentication. Create a login function.”

IMPETUS

3. **Role-Based Prompts**

- Tell the AI to behave like a specific expert.
- **Example:**
- “Act as a senior Java developer. Optimize the following code.”

4. Question-Based Prompts

- Ask questions to get explanations or information.
- **Example:**
- “What is the difference between REST and GraphQL?”

5. Example-Based Prompts (Few-Shot Prompts)

- Provide examples to show the pattern.
- **Example:**
- Input → “Hello”
- Output → “H-E-L-L-O”
- Now translate “World” the same way.
-

6. Chain-of-Thought Prompts

- Ask AI to explain step-by-step reasoning.
- **Example:**
- “Explain your reasoning step-by-step before giving the final answer.”

IMPETUS

7. Code Generation Prompts

- Used for generating new code.
- **Example:**
- “Generate a Node.js API endpoint for user login using JWT.”

8. Code Fixing / Debugging Prompts

- Used to find and solve errors in code.
- **Example:**
- “Fix the bug in this function. It crashes when input is empty.”

9. Summarization Prompts

- Ask AI to summarize content.
- **Example:**
- “Summarize this article in 3 bullet points.”
-

10. Transformation Prompts

- Convert data from one format to another.
- **Example:**
- “Convert this JSON into a SQL insert query.”

IMPETUS

How AI Assists in Early SDLC Phases

- AI accelerates and improves early stages of software development by providing:
- Faster documentation
- Better clarity
- Automated scaffolding
- Architecture suggestions
- Early validation of requirements

IMPETUS

Requirement Analysis & Planning

- **How AI Helps:**
- Extracts key requirements from documents or client conversations
- Identifies ambiguities, missing details, contradictions
- Suggests additional edge cases and scenarios
- Helps estimate complexity and effort
- **Example Prompt:**
- “Analyze the following BRD and list functional, non-functional requirements and dependencies.”

IMPETUS

Generate User Stories & Acceptance Criteria

- **AI Generates:**
  - **User Stories (As a… I want… So that…)**
  - **Acceptance Criteria** (Given / When / Then)
  - **Epics & Tasks**
- **Benefits:**
- Standardized structure
- Faster creation
- Better backlog grooming
- **Example:**
- “Create 10 user stories and acceptance criteria for an online food delivery module.”

IMPETUS

Converting Business Requirements to Technical Specs

- AI can convert high-level requirements into:
- System workflows
- API specifications
- Sequence diagrams
- Module decomposition
- Business logic documentation
- **Example:**

"Convert these business rules into a technical specification with API endpoints and data flow."

IMPETUS

Architecture & Design Support

- AI assists architects and engineers in planning the system design.
- 🔹 **Suggested Architecture:**
- Monolith vs Microservices
- Event-driven architecture
- CQRS, DDD patterns
- API Gateway & Integration patterns
- **Example Prompt:**
- “Suggest a microservices architecture for an e-commerce checkout service with scalability and fault-tolerance.”

IMPETUS

Generate Design Patterns & UML Diagrams

- AI can generate text-based representations of UML, such as:
- Class Diagrams
- Sequence Diagrams
- Activity Diagrams
- Component Diagrams
- **Example:**
- “Generate a UML class diagram for a Library Management System.”
- (You can convert these using tools like Mermaid, PlantUML, or Draw.io)

IMPETUS

Database Design Using DBML

- AI can produce DBML syntax for:
- Table definitions
- Relations
- Constraints
- Indexes
- **Example Prompt:**
- “Create a DBML schema for a hotel booking system with Users, Rooms, Bookings tables.”
- This DBML can be imported into:
- dbdiagram.io
- DBeaver
- DrawSQL

IMPETUS

Tool Usage in Early SDLC

- **Popular Tools:**
- **GitHub Copilot** – Requirements → Code suggestions
- **Cursor AI** – Project scaffolding + architectural analysis
- **ChatGPT / Claude** – Requirement & architecture automation
- **Windsurf** – Real-time coding and refactoring
- **Mermaid/PlantUML** – Diagram generation

IMPETUS

Using Copilot/Cursor to Scaffold Project Structure

- **AI Can Generate:**
- Folder structure
- Boilerplate code
- API route templates
- Configuration files
- DevOps YAML files
- Test setup files
- **Example Prompt:**
- “Create a production-ready folder structure for a Node.js + Express + MongoDB application following clean architecture.”
- **Benefits:**
- Saves time
- Enforces clean structure
- Helps junior developers onboard faster

IMPETUS

Case Study

AI-assisted development of an E-Commerce Backend using Spring Boot microservices

IMPETUS

Our Project

- Built an e-commerce backend using four Spring Boot microservices: Auth, Product, Order, and Inventory.
- Used PostgreSQL with Liquibase-managed schema updates for consistent database setup across services.
- Added JWT-based authentication, REST APIs, Swagger documentation, and a Postman collection for quick testing.
- Packaged local execution with Docker Compose so the full platform can start with database, migrations, services, and Swagger landing page.

IMPETUS

What We Did

- **Implementation highlights**
- Generated the initial service structure, entities, controllers, and repository layers faster with GenAI support.
- Connected Order Service with Product and Inventory services for stock validation and order total calculation.
- Added Actuator endpoints, validations, and cleaner exception handling to make the services more production-ready.

- **Manual refinements after AI output**
- Replaced deprecated Spring Security patterns with SecurityFilterChain for Spring Boot 3 compatibility.
- Fixed database naming issues such as reserved table names by aligning entities with Liquibase schema design.
- Expanded testing and documentation so the project is easier to run, validate, and present as a complete backend solution.

IMPETUS

How We Used Prompts and GenAI

- Architecture prompt: "Design a Spring Boot e-commerce backend using auth, product, order, and inventory microservices with PostgreSQL."
- Scaffolding prompt: "Generate a production-ready folder structure, entities, controllers, DTOs, and CRUD APIs for each service."
- Refinement prompt: "Update this code for Spring Boot 3 and Spring Security 6, and avoid reserved SQL keywords in table mapping."
- Debugging prompt: "Add inter-service communication so order creation checks inventory and calculates totals before saving."
- Documentation prompt: "Create README, API summary, and setup instructions for Docker Compose, Liquibase, and Swagger endpoints."
- Impact: prompts reduced setup time, accelerated boilerplate generation, and let us focus manual effort on business logic, compatibility fixes, and quality improvements.

Thanks.

IMPETUS